Session 07 — Homework

Finish What Class Didn't Get To

Complete before: Session 08 **Time you need:** about 45–60 minutes for CORE

Before you start

The instructor published a few small corrections after class (styling for the section headings and people summary, a page-title cleanup, and some CSS cleanup). **You are starting from the instructor's latest Session 07 state, not the copy you had at the end of class** — bring those updates into your personal branch first, using the same workflow from the Git & GitHub Handbook.

```
git checkout <your-student-branch>
git status
```

If `git status` shows changes you have not committed yet, commit those changes first. Do not `fetch` / `merge` the group update while you still have uncommitted work.

When your working tree is clean:

If you're in Group 1:

```
git fetch
git merge origin/group-1
```

If you're in Group 2:

```
git fetch
git merge origin/group-2
```

If Git reports a conflict, resolve it the normal way (see the handbook's "Merge conflicts" section) — this is expected and not an error.

Then run `npm run dev` and confirm it works with no console errors before you begin.

Class ran out of time before finishing the person-filter buttons, so that work is now homework instead of teamwork. Every exercise below uses only concepts already taught through Session 07 (`.map()`, `.filter()`, arrow functions, derived data, conditional rendering). **No stateful task**

collection. No Add/Toggle/Delete/Edit. No callback props. No spread. No `useEffect`. No destructuring. No Fragment. `tasks` stays exactly what it is tonight: a plain `const` array.

When you're done

Homework happens directly on your own personal branch — the same branch you already used above.

Validate: `npm run build` **Commit:** `git add .` then `git commit -m "Session 07 homework"` **Push:** `git push`

CORE — everybody does this

Goal

Make the named person-filter buttons actually work.

What already works

- `selectedUserId` already exists as state, and `visibleTasks` already checks it — you built that live tonight.
- The "All people" button already works: clicking it resets `selectedUserId` to `0`.
- Each named person button (one per person) already renders with the right name and a `key`. It currently does nothing when clicked.

What to build

1. A normal, named function that takes a person's `id` and updates `selectedUserId` to that id.
2. Wire every named person button's `onClick` so clicking it calls that function with *that specific button's* user id.
3. Make the currently-selected named person's button show the same "active" look the status filter buttons and "All people" already use — and only that one button.

Rules

- Reuse the exact `useState` setter pattern you already used for every other filter this session.
- You already know normal event handlers, and you learned arrow functions today. For this task, you need to combine those ideas so each person's button can provide the correct id when it

is clicked. Think about *why* a plain `onClick={handleSomething(user.id)}` would be wrong here, the same mistake from Session 06, just with an argument this time.

- The "active" class logic already exists for the status buttons and for "All people" — look at how those are built before writing your own version for the named buttons.
- Don't touch `visibleTasks` — the three-way filter (status + search + person) already reads `selectedUserId` correctly. If your button doesn't change the visible list, the bug is in your `onClick`, not in the filtering logic.

Test it

- Click each named person's button — only that person's tasks should show, and only that button should look active.
- Click "All people" — everything should reset correctly.
- Combine a named person with a status filter and with search — all three should keep working together.

Done when

- □ Clicking any named person's button shows only that person's tasks.
- □ Exactly one button (the selected person, or "All people") looks active at a time.
- □ Status + search + person filters still combine correctly.
- □ `npm run build` succeeds with zero errors.

Hint

You've already built this exact interaction shape once tonight, for a button that didn't need an argument. The only new part is passing the right id to the handler.

STRETCH — if CORE was comfortable

Goal

Show a real message when no tasks match the current filters.

What already works

Right now, if a status/search/person combination matches nothing, the task list area is just empty — no rows, no message.

What to build

When `visibleTasks` has zero items, show this text instead of the task list:

> No tasks to show.

When `visibleTasks` has one or more items, show the task list exactly as it works today.

Rules

- You already know this exact shape from Session 06: check a condition, render one thing or another.
- A CSS class called `.empty-state` already exists in your stylesheet, instructor-provided — use it on whatever element shows the message. You do not need to write any new CSS.
- Don't change how tasks are filtered — only change what renders when the result is empty.

Test it

Try combinations of status, search, and person until you find one that matches zero tasks (for example, search for something that doesn't exist). Confirm you see the message instead of a blank area. Then clear your filters and confirm the task list comes back correctly.

Done when

- □ A filter/search/person combination matching zero tasks shows "No tasks to show."
- □ Any combination matching one or more tasks still shows the task list normally.
- □ `npm run build` succeeds with zero errors.

Hint

You already have a value that tells you exactly how many tasks are currently visible.

CHALLENGE — for students who want more

Goal

Create one reusable people-with-counts list, and use it in both the people summary and the person-filter buttons.

What already works

Right now, the people summary calculates "how many tasks does this person have" inline, inside its own `.map()` — it's the only place that relationship (which person owns which tasks) is calculated. The person-filter buttons don't show a count at all yet. If you added one there the obvious way, you'd end up calculating that same relationship a second time, separately, in a different `.map()`.

What to build

Derive **one** value — a list that pairs each person with their real task count, keeping only people who have at least one task. Build it once, then use that *same* value to render both the people summary and each person-filter button's label (name + count) — instead of the people summary calculating its own count inline while the buttons calculate it again separately.

Rules

- Use `.map()` and `.filter()` — both already familiar.
- Build this derived value once, near your other derived values (`visibleTasks`, `totalCount`, etc.) — not inside JSX.
- The people summary should end up reading from this one value instead of doing its own separate calculation.
- No new state. This is still something calculated fresh every render, like everything else this session.

Test it

Confirm the people summary still shows the correct counts, and confirm each person-filter button now also shows a number next to their name. Change nothing about who currently has zero tasks — they should still be excluded from the summary the same way as before.

Done when

- □ One derived value pairs each person with their task count.
- □ Both the people summary and the person-filter buttons read from that same value.
- □ Each person-filter button shows their name and their real task count.
- □ `npm run build` succeeds with zero errors.
- □ No stateful task collection, CRUD, callback props, spread, `useEffect`, or destructuring appears anywhere in your solution.

One question to think about — no code required

Every piece of data tonight — tasks, users, filters — has been read-only. Nothing you built can actually add, remove, or check off a task.

What would have to change about `tasks` for a button to actually mark a task as completed?

Write your answer in a sentence or two. You do not need to implement it — that's exactly what Session 08 is about.